

What is an Array?

An **Array** is a linear collection of elements of the same data type stored in contiguous (continuous) memory locations. It is also known as a **subscript variable**.

Why Do We Need Arrays?

Suppose you need to write a program to calculate the average of 100 numbers.

The Regular Method: You would have to declare 100 separate variables (int n1, n2, n3... n100;). Managing, reading, and updating 100 different variable names makes your code incredibly lengthy, confusing, and difficult to maintain.

The Array Method: You declare a single array variable with a size of 100 (int numbers[100];). All elements share the same name and are accessed easily using an index number.

1. One-Dimensional (1-D) Arrays

A. Declaration Syntax

To declare a 1-D array, use the following syntax structure:

```
data_type array_name [size];
```

data_type: The type of data elements that will be stored inside the array (e.g., int, float, char).

array_name: Any valid C identifier name.

size: A positive natural number enclosed in square brackets [] specifying how many total elements the array can hold.

Example: int age[100]; creates an integer array named age that allocates room for exactly 100 individual whole numbers.

B. Element Access & The Subscript Operator ([])

Array elements are manipulated using an index (or subscript) value. In C, **array indexing always starts at 0** and ends at size - 1.

Given: int a[100];

Indices run from: a[0], a[1], a[2], a[3] ... up to a[99]

Accessing elements: `a[3] = 26;` assigns the value 26 to the 4th element of the array.

Here, `[]` acts as the **subscript operator**, while `a` and `3` function as its operands.

C. Critical Rules for 1-D Array Declarations

Size Cannot Be Empty: You cannot leave the size brackets empty during a standalone declaration.

✗ `int a[ ];` ----> **Compilation Error**

Sequential Memory Allocation: No matter how large an array is, it always consumes memory in a **continuous, sequential block**.

The Garbage Value Trap: When a local array (declared inside a function) is left uninitialized, its element blocks will contain completely unpredictable **garbage values**.

Complete Initialisation: You can initialize array during declaration

`int a[5] = {10, 50, 30, 70, 20};`

Array Overflow: You cannot initialize an array with more values than its explicitly declared size allows.

✗ `int a[5] = {10, 50, 30, 70, 20, 80, 40};` ----> **Compilation Error**

Partial Initialisation: If you initialize an array with *fewer* values than its specified size, the compiler populates the remaining slots automatically with **0** (not garbage values).

`int a[5] = {10, 50};` ----> **Resulting memory layout:** `[10, 50, 0, 0, 0]`

Implicit Sizing: You can leave the size brackets `[]` empty **only if** you initialize the array items concurrently on the same line. The compiler automatically calculates the correct size based on the element count.

`int a[] = {10, 50, 30, 80, 20};` ----> **Allocates an array of size 5.**

2. Two-Dimensional (2-D) Arrays (Matrices)

A **Two-Dimensional Array** is an "Array of Arrays". It maps data structurally into a grid interface consisting of **rows** and **columns**, commonly known as a **Matrix**.

A. Syntax Specification

`data_type array_name [r][c];`

r: Specifies the total number of horizontal **rows**.

c: Specifies the total number of vertical **columns**.

Total Elements Capacity = $r * c$

Example: `int arr[3][2];` creates a matrix grid containing 3 rows and 2 columns, totaling 6 elements.

B. Structural Rules for 2-D Initialisation

When declaring and initializing a 2-D array simultaneously, **specifying the second dimension (columns) is strictly mandatory**, while the first dimension (rows) remains optional.

Correct Variants:

```
int arr[2][3] = {{5, 8, 9}, {10, 12, 11}};
```

```
int arr[][3] = {12, 34, 56, 78, 91, 23}; (Compiler automatically  
determines it needs 2 rows)
```

✗ Incorrect Variants (Compilation Errors):

```
int arr[2][];
```

```
int arr[][];
```

C. Implementation Blueprint: Printing a Matrix

To navigate across the row and column architecture of a 2-D array, we must use **nested loops**.

```
#include <stdio.h>
int main()
{
    int a[2][3] = { {5, 8, 9}, {10, 12, 11} }; ->Declares a matrix with 2 rows and 3 columns
    int i, j;
    for(i = 0; i < 2; i++) ----> Outer loop controls Rows
    {
        for(j = 0; j < 3; j++) ----> Inner loop controls Columns
        {
            printf("%d ", a[i][j]); ----> Prints elements side-by-side
        }
        printf("\n"); ----> Jumps to next line after completing a full row
    }
    return 0;
}
```

Output:

```
5 8 9
10 12 11
```

3. Three-Dimensional (3-D) Arrays

A **Three-Dimensional Array** is an "Array of 2-D Arrays" (or an array of arrays of arrays). Think of it like a multi-page book, where each page contains an independent 2-D grid matrix.

A. Syntax Specification

```
data_type array_name [pages][rows][columns];
```

Example: `int a[2][3][2];` allocates a data layout consisting of 2 complete matrix blocks, where each block contains 3 rows and 2 columns.

B. Implementation Blueprint: Accessing a 3-D Space

To iterate completely over a 3-D array block, we require **three nested loops** to drive through pages, rows, and columns sequentially.

```
#include <stdio.h>
int main()
{
    int a[2][3][2] = {
        { {4, 5}, {6, 7}, {8, 0} }, ----- > Matrix Page 0
```

```

    { {11, 12}, {13, 0}, {0, 0} } -----> Matrix Page 1
};
int i, j, k;

for(i = 0; i < 2; i++) -----> Loop 1: Controls the Page Layers
{
    for(j = 0; j < 3; j++) -----> Loop 2: Controls the Rows on the current page
    {
        for(k = 0; k < 2; k++) -----> Loop 3: Controls the Columns in the current row
        {
            printf("%d ", a[i][j][k]);
        }
        printf("\n"); -----> Line break after a row completes
    }
    printf("\n"); -----> Extra line break after a full page layer completes
}
return 0;
}

```

Summary Memory Layout Reference

Array Dimension	Index Reference	Access Analogy	Structural Purpose
1-D Array	a[i]	A single row of post office boxes	Linear list tracking
2-D Array	a[i][j]	A flat spreadsheet page grid	Mathematical Matrix computing
3-D Array	a[i][j][k]	A multi-page collection of spreadsheets	Complex spatial data tracking

Stuck or Prefer a Visual Walkthrough?

If you are not able to understand through these notes or you need to explore these concepts more visually, you can review my handwritten notes PDF provided in the same folder: **10 Array** .